B.TECH MAJOR PROJECT — MILESTONE 1 EVALUATION (MARKS: 30)

# MovieMind: An Explainable and Conversational Movie Discovery Platform


| | | | |
|---|---|---|---|
| **Course:** | B.Tech Capstone Project | **Department:** | Computer Science & Engineering |
| **Team Members:** | Rajat Verma (E23CSEU1490)<br>Alok Verma (E23CSEU1489)<br>Yash Bharadwaj (E23CSEU1499) | **Submission Date:** | October 18, 2026 |
| **Prototype URL:** | moviemind-eight.vercel.app | **Repository:** | github.com/rajatverma777/MovieMind |


---

## 1. Final Problem Statement and Motivation

### 1.1 Problem Statement

In modern home entertainment, the rapid expansion of subscription streaming services (Netflix, Prime Video, Disney+ Hotstar, Apple TV+) has caused extreme decision fatigue. Viewers spend between 15 to 20 minutes per viewing session browsing catalogs without reaching a decision.

This problem is compounded by four specific architectural limitations:

1. **Catalog Siloing:** Commercial engines are strictly confined to their own licensed libraries and will never suggest matching content hosted on competing platforms.
2. **Algorithmic Opacity (The Black-Box Dilemma):** Recommenders utilize complex matrix factorization or deep networks with zero mathematical explainability. Users are offered content without understanding the criteria behind the selection.
3. **Rigid Interfaces & Chatbot Hallucinations:** Standard search bars require exact spelling or simple dropdown tags, failing on conversational nuance (e.g., *"a rainy psychological thriller with a melancholic mood"*). Conversely, general LLMs (like ChatGPT) frequently hallucinate non-existent movie titles, mix up sequels, and provide inaccurate release years.
4. **Fragmented Trailer Discovery & Lack of Unified Video Previews:** Currently, movie trailers and teasers are scattered across external video sites and distributor channels. When discovering a movie, viewers must leave the app, open YouTube, dodge clickbait cuts and spoilers, and switch back and forth.

### 1.2 Motivation

**MovieMind** bridges this gap by unifying cross-platform discovery, mathematical explainability, and conversational assistance into a single hub. By combining a 19-dimensional genre vector space model (Cosine Similarity) with embedded high-definition official trailers and an orchestrated Google Gemini AI assistant, MovieMind delivers transparent recommendations with verified metadata and zero hallucinations.

## 2. UN Sustainable Development Goals (SDG) Alignment

| TARGET SDG | SPECIFIC TARGET | MOVIEMIND ARCHITECTURAL RATIONALE |
|---|---|---|
| **SDG 9: Industry, Innovation, and Infrastructure** | Target 9.5: Enhance scientific research and technological innovation | Demonstrates a decoupled hybrid AI paradigm combining mathematical vector algebra with modern generative models deployed on efficient, low-cost serverless microservices. |
| **SDG 12: Responsible Consumption and Production** | Target 12.8: Ensure relevant information for sustainable lifestyles | Reduces digital compute and bandwidth waste caused by prolonged, fruitless 20-minute user searches. Promotes privacy-preserving client-side data storage without centralized data harvesting. |
| **SDG 4: Quality Education & Information Literacy** | Target 4.4: Increase digital literacy and transparent technology | Demystifies recommendation black boxes by displaying explicit Cosine Similarity match percentages (e.g., 82%) and genre convergence reasons to the user. |

## 3. Background, Context & Literature Review (Recent Research Included)

Recommender Systems (RS) have evolved from classical Matrix Factorization to Content-Based Vector Modeling and modern Tool-Augmented Conversational LLMs. Recent academic literature underscores the necessity of combining generative language models with deterministic domain engines:

| STUDY / AUTHOR | FOCUS AREA | KEY FINDINGS & STRENGTHS | IDENTIFIED LIMITATIONS ADDRESSED BY MOVIEMIND |
|---|---|---|---|
| **Salton & McGill (1983)** | Vector Space Model (VSM) | Mathematically proven, bounded ($[0,1]$), low computational complexity $O(n)$ for similarity calculation. | Operates on static feature arrays; lacks natural language dialogue capabilities. |
| **Adomavicius & Tuzhilin (2005)** | Content-Based Filtering | Solves collaborative filtering cold-start problems; high item independence without tracking user graphs. | Requires structured taxonomies; lacks conversational preference elicitation. |
| **Fan et al. (2023)** ***IEEE TKDE*** | LLMs in Recommender Systems | Proves that LLMs excel as conversational orchestrators when strictly paired with external deterministic search/vector tools. | Pure LLM prompting without external tools suffers from high token latency and unverified suggestions. |
| **Hou et al. (2024)** ***ACM TOIS*** | Zero-Shot LLM Evaluation | Comprehensive benchmark of LLM recommendations; confirms high semantic intent parsing from user queries. | Discloses critical **popularity bias and hallucinations** in LLMs unless grounded by verified API catalogs. |
| **Zhang & Chen (2023)** ***Foundations & Trends in IR*** | Explainable AI (XAI) in RS | Demonstrates that feature-level explainability (e.g., genre overlap %) significantly boosts user trust and adoption. | Black-box neural embeddings hide the rationale behind suggestions, degrading user trust. |
| **Gomez-Uribe & Hunt (2015)** ***ACM TMIS*** | Netflix Industrial Recommender | Highly effective deep matrix factorization driving 80%+ of streams within a single streaming provider. | Walled garden (only Netflix titles); complete black-box; lacks unified cross-platform discovery. |

## 4. Gap Analysis

> **The MovieMind Competitive Advantage:** Unlike closed streaming platforms that only recommend internal content, and unlike standalone LLMs (like vanilla ChatGPT) that hallucinate non-existent titles, MovieMind couples real-time TMDB metadata with a mathematically grounded Cosine Similarity engine and conversational tool-calling agent.

## 5. Stakeholders & Requirements Engineering

### 5.1 Functional Requirements (FR)

- **FR-01 (Live Catalog Retrieval):** Dynamic retrieval of trending, top-rated, and now-playing films from TMDB REST API.
- **FR-02 (Title & Keyword Search):** Real-time keyword search with instant query results and fuzzy matching.

- **FR-03 (Unified Trailer Playback):** Direct high-definition official trailer playback embedded in the movie details modal.
- **FR-04 (Client Watchlist Persistence):** Bookmarkable personal watchlist persisted locally via browser Web Storage (LocalStorage).
- **FR-05 (19-Dim Vector Modeling):** Automatic formulation of 19-dimensional genre vectors matching standard TMDB categories.
- **FR-06 (Cosine Similarity Scoring):** Real-time vector dot product ranking against the user's taste profile centroid.
- **FR-07 (Conversational Dialogue):** Multi-turn natural language conversational chat with structured card rendering.

### 5.2 Non-Functional Requirements (NFR)

- **NFR-01 (Performance & Latency):** Vector math computation latency < 100ms; total conversational response < 3.0s.
- **NFR-02 (Security & Isolation):** Zero API key leakage; all secret keys isolated on serverless environment variables.
- **NFR-03 (Fault Tolerance):** Automatic failover between primary Gemini Flash and secondary fallback models.
- **NFR-04 (Privacy by Design):** Client-side taste storage with zero invasive tracking or telemetry logging.

## 6. Technology Stack & Tool Selection

| LAYER | TECHNOLOGY SELECTED | TECHNICAL JUSTIFICATION |
|---|---|---|
| **Frontend** | React 18 + Vite | Ultra-fast virtual DOM diffing, modern component state, lightweight bundle (< 100kB). |
| **Backend API** | Node.js & Vercel Serverless | Asynchronous event loop, instant serverless spin-up, zero idle hosting costs. |
| **Knowledge Base** | The Movie Database (TMDB) | 800,000+ verified film records, standardized 19-genre taxonomies, trailer video endpoints. |
| **AI Model** | Google Gemini (3.5 Flash / 2.5 Flash Lite) | High inference speed, native structured JSON schema compliance, low latency. |
| **Agentic Orchestration** | LangChain (`@langchain/core`) | Reliable tool-calling pipeline, prompt template isolation, and structured validation via Zod. |

## 7. 4-Way Feasibility Analysis

- **Technical Feasibility:** High. Proven mathematical foundation (Cosine Similarity runs in microseconds); verified Google GenAI and TMDB SDK integration.

- **Economic Feasibility:** High. Designed for zero-cost operation using Vercel Hobby tier, TMDB Free Developer API, and Gemini free-tier quotas.
- **Operational Feasibility:** High. Single-page web application requiring zero native client installation, accessible on all modern browsers and smartphones.
- **Legal / Ethical Feasibility:** High. Fully compliant with TMDB API terms; does not host infringing video files; prioritizes user privacy through client-side state.

## 8. Scope, Risk Analysis & Mitigation

| RISK CATEGORY | POTENTIAL IMPACT | ENGINEERED MITIGATION STRATEGY |
|---|---|---|
| **API Rate Limiting (HTTP 429)** | Chat stalls if Gemini quota is exceeded | Automated multi-tier failover: Primary `gemini-3.5-flash` retries with `gemini-2.5-flash-lite`, and ultimately falls back to the deterministic engine. |
| **Network Latency** | Slow candidate pool loading from TMDB | 15-minute in-memory LRU candidate cache on backend to prevent repeated API calls. |
| **LLM Hallucinations** | Recommending non-existent movies | Tool-calling architecture: The LLM is restricted from generating movie lists directly; it must query the verified TMDB recommendation tool. |

## 9. Proposed Solution and Detailed Methodology

### 9.1 Five-Stage Engineering Methodology

STAGE 1

**Data Ingestion**

Real-time REST API fetching from TMDB (/trending, /now_playing, /videos). Payload normalization into standardized JSON records.

STAGE 2

**Vector Modeling**

Mapping categorical movie genres into 19-dimensional binary vectors V_M ∈ {0,1}^19 corresponding to TMDB taxonomy.

STAGE 3

**Cosine Similarity**

Formulating user taste centroid U across watchlist films. Calculating normalized dot product similarity against candidate pools.

STAGE 4

**AI Orchestration**

Google Gemini Flash orchestrated via LangChain tool calling (recommend_movies, search_movies) with Zod structured output.

STAGE 5

**Media Previews**

Embedding official YouTube HD trailers directly into movie modal. Client-side LocalStorage persistence for privacy sovereignty.

VERIFICATION

**Explainability**

Displaying exact Cosine Similarity percentage (e.g., 82% Match) alongside natural language reasons grounded in genre DNA.

### 9.2 Mathematical Formulation (Cosine Similarity Engine)

Each film is mapped to a 19-dimensional binary vector $\vec{V}_M$ matching standard TMDB genre IDs. The user profile $\vec{U}$ is computed as the normalized centroid of liked films. The similarity score is computed as:

$$\text{Cosine Similarity}(U, V) = ( U \cdot V ) / ( ||U|| \times ||V|| ) = \sum(U_i \times V_i) / ( \sqrt{\sum(U_i)^2} \times \sqrt{\sum(V_i)^2} )$$

Candidate films are sorted in descending order of similarity ($S \in [0.0, 1.0]$), filtering out titles already consumed by the user.

### 9.3 Flow Diagram 1: How the Website Works (End-to-End User Interaction Flow)

**WEBSITE INTERACTION & USER FLOW**

**Step 1** **User Opens Platform:** Landing page initializes and immediately queries TMDB REST endpoints for live Weekly Trending and Now Playing movies.

▼

**Step 2** **Interactive Navigation:** User can either browse dynamic genre carousels, search via keyword/title, or launch the Conversational AI Chat.

▼

**Step 3** **Movie Detail & Unified Trailer Preview:** Clicking any film opens a modal with synopsis, cast, ratings, and embedded HD trailer playback without leaving the page.

▼

**Step 4** **Taste Vector Personalization:** Adding movies to the Watchlist updates the user's 19-dimensional taste centroid vector in browser LocalStorage.

### 9.4 Flow Diagram 2: Conversational AI & Dual-Engine Recommendation Flow

### 🤖 AI AGENT & MATHEMATICAL VECTOR PIPELINE

**User Query** User inputs natural language prompt: *"Recommend 2 sci-fi movies like Interstellar"*

▼

**LangChain Agent** Node.js backend sends prompt + conversation history to **Google Gemini Flash** via LangChain tools.

▼

**Tool Calling** Gemini identifies recommendation intent and invokes `recommend_movies(referenceTitle: "Interstellar")`.

▼

**Cosine Engine** Calculates dot-product similarity against in-memory candidate movie pool and returns top-ranked matches.

▼

**Structured UI Output** Gemini formats response into verified JSON cards displaying poster artwork, match percentages, and explanations.

## 10. Work Completed & Two-Milestone Roadmap

In accordance with Milestone 1 guidelines, the project proposal, feasibility analysis, architecture design, and working UI prototype are complete. The remaining development is organized into Milestone 2:

| MILESTONE PHASE | WORK PACKAGE / MODULE | SCOPE & KEY DELIVERABLES | EXECUTION STATUS |
|---|---|---|---|
| **Milestone 1** | **Problem Formulation & Literature Review** | Define core challenges (catalog siloing, opaque algorithms, trailer fragmentation). | Completed |
| **Milestone 1** | **Requirements & Feasibility Analysis** | Establish Stakeholders, 8 Functional & 5 Non-Functional Requirements, and 4-way feasibility report. | Completed |
| **Milestone 1** | **Base UI & TMDB Integration Prototype** | Build responsive React 18 single-page application, TMDB API movie search, and local watchlist bookmarking. | Completed |
| **Milestone 1** | **Milestone 1 Proposal & Defense** | Formal Capstone Milestone 1 document preparation, SDG mapping, and academic panel evaluation defense. | Active (Current) |
| **Milestone 2** | **Recommendation Engine Implementation** | Mathematical modeling of 19-dimensional genre vectors, user taste centroid calculation, and Cosine Similarity ranking. | Scheduled |
| **Milestone 2** | **Backend Proxy & Candidate Caching** | Node.js / Express serverless API endpoints, environment variable key isolation, and candidate pool in-memory caching. | Scheduled |
| **Milestone 2** | **Conversational AI & LangChain Tools** | Integration of Google Gemini Flash via LangChain tool calling (`recommend_movies`, `search_movies`) with structured card output. | Scheduled |
| **Milestone 2** | **Testing, Benchmarking & User Trials** | End-to-end latency measurement (< 3s target), recommendation relevance evaluation, and user testing. | Scheduled |
| **Milestone 2** | **Final Dissertation & Capstone Defense** | Final code freeze, project documentation, dissertation submission, and presentation before the evaluation committee. | Scheduled |

## 11. References (Academic Citations)


1. **Salton, G., & McGill, M. J. (1983).** *Introduction to Modern Information Retrieval.* McGraw-Hill, New York.
2. **Adomavicius, G., & Tuzhilin, A. (2005).** *Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions.* IEEE Transactions on Knowledge and Data Engineering, 17(6), 734–749.
3. **Ricci, F., Rokach, L., & Shapira, B. (2015).** *Recommender Systems Handbook.* Springer Science+Business Media, New York.
4. **Fan, W., Zhao, Z., Li, J., Liu, H., Mei, Q., & Wang, Q. (2023).** *Recommender systems in the era of large language models (LLMs).* IEEE Transactions on Knowledge and Data Engineering (TKDE), 36(6), 2840–2858.
5. **Hou, Y., Zhang, J., Lin, Z., Lu, H., Chen, X., & Zhao, W. X. (2024).** *Large language models are zero-shot recommenders: An exploratory study.* ACM Transactions on Information Systems (TOIS), 42(3), 1–29.
6. **Zhang, Y., & Chen, X. (2023).** *Explainable recommendation: A survey and new perspectives.* Foundations and Trends® in Information Retrieval, 14(1), 1–101.

7. **Gomez-Uribe, C. A., & Hunt, N. (2015).** *The Netflix recommender system: Algorithms, business value, and innovation.* ACM Transactions on Management Information Systems (TMIS), 6(4), 1–19.
8. **Koren, Y., Bell, R., & Volinsky, C. (2009).** *Matrix factorization techniques for recommender systems.* Computer, 42(8), 30–37.
9. **LangChain Documentation (2024).** *Agents, Tool Calling, and Structured Outputs in JavaScript.* https://js.langchain.com/
10. **Google DeepMind (2024).** *Gemini: A Family of Highly Capable Multimodal Models.* Google Technical Report.
11. **The Movie Database (TMDB) API (2024).** *API Documentation and Developer Guides.* https://developer.themoviedb.org/docs
12. **United Nations (2015).** *Transforming our world: the 2030 Agenda for Sustainable Development.* Resolution adopted by the General Assembly, A/RES/70/1.